ASSIGNMENT 9.2

Wine Quality API v1.1

Hacer cumplir, documentar y probar el contrato de S7 sin romper a sus clientes

DURACION	MODALIDAD	PREREQUISITOS
120 min	Individual, taller guiado por tests	Assignment 9.1 y API funcional de S8

1. Encargo

Llevar la Wine Quality API de 1.0.0 a 1.1.0 completando los TODO del starter en orden: salida tipada y filtrada, errores con un único sobre, OpenAPI útil para quien consume y tests propios que demuestren el contrato.

Regla de continuidad
El contrato público de S7 no cambia: POST /v1/predictions, GET /health y el sobre de error con code, message, fields y request_id se hacen cumplir y se documentan, no se rediseñan.

2. Punto de partida

- El starter es la API de S8: valida la entrada con Pydantic y carga el predictor una vez en el lifespan, pero devuelve dicts y errores {"detail": ...}.

- Al empezar, uv run pytest -q da 4 tests en verde y 23 en rojo; cada test en rojo corresponde a un TODO S9.

- Checkpoints: 8 en verde tras S9.1, 16 tras S9.2, 23 tras S9.3 y la suite completa tras S9.4.

3. Material que recibes

- exercises/02-wine-api-hardening/problem/starter/ con su propio uv.lock y los TODO S9.1-S9.4

- Suite de aceptación de S9.1-S9.3 como especificación ejecutable

- predictor.py, request_id.py y dependencies.py terminados; export_openapi.py para el snapshot

- WINE_API_MODE=unavailable para simular un modelo no cargado

4. Tareas

01 Arranca y mapea - Sincroniza el starter, confirma 23 tests en rojo y 4 en verde y localiza los TODO con grep -rn "TODO S9" src tests antes de editar.

02 S9.1 - Contrato de salida - Declara en Prediction solo los cuatro campos públicos del contrato; decide en el contrato, no en el predictor, cómo se redondea confidence y cómo se escribe served_at en UTC con sufijo Z. Las rutas devuelven modelos tipados.

03 S9.2 - Errores - Implementa y registra los manejadores de validación, WineApiError, HTTPException de Starlette y error inesperado; /health lanza ModelUnavailableError. Cada fallo recibe su status y error.code sin filtrar detalles internos.

04 S9.3 - OpenAPI - Completa los metadatos de la app, versión incluida; prefix, tags y responses 400, 422 y 503 en el APIRouter; description con unidad en cada Field; summary, docstring y al menos tres ejemplos de cuerpo en Swagger.

05 S9.4 - Tests propios - Sustituye los cuatro pytest.fail: 503 con dependency_overrides y limpieza, cinco o más casos fuera de contrato con parametrize, límites inclusivos de alcohol y snapshot de OpenAPI.

06 Versiona y reúne evidencia - Exporta y versiona contracts/openapi.json, deja pytest y Ruff en verde, captura /docs y redacta la nota de compatibilidad.

5. Entrega

Evidencia	Que debe permitir comprobar
API 1.1.0	TODO resueltos y uv run pytest -q sin fallos.
contracts/openapi.json	Snapshot generado con wine_api.export_openapi y versionado en Git.
Calidad	uv run ruff check . y uv run ruff format --check . en verde.
Captura de /docs	Predicción desplegada con errores 400, 422 y 503 y ejemplos de cuerpo.
Nota de compatibilidad	5-10 líneas: qué cambio es compatible, cuál rompería a los clientes de S7 y cómo lo versionarías.

6. Criterios de aceptacion

[ ] raw_score no aparece en ninguna respuesta ni en contracts/openapi.json.

[ ] confidence sale con dos decimales y served_at en UTC, ISO 8601 y con sufijo Z.

[ ] Ningún error devuelve {"detail": ...}, trazas, rutas de ficheros, texto de excepción ni input.

[ ] Las rutas lanzan errores de dominio y los manejadores deciden el status; no hay HTTPException en el dominio.

[ ] /docs documenta 400, 422 y 503 con ErrorResponse, tags en el router y tres ejemplos de cuerpo.

[ ] Los tests propios usan dependency_overrides, parametrize y snapshot sin red real; los archivos protegidos quedan intactos.

7. Rubrica orientativa

Criterio	Puntos
Contrato de salida tipado y filtrado (S9.1)	2,5
Errores consistentes y seguros (S9.2)	3
OpenAPI útil para un consumidor (S9.3)	2
Tests propios con intención (S9.4)	2
Calidad y nota de compatibilidad	0,5

8. Comprobacion

```
# Preparación
cd semana9/modules/09-api-professional/exercises/02-wine-api-hardening/problem/starter
uv sync --locked
uv run pytest -q
grep -rn "TODO S9" src tests

# Checkpoint por paso
uv run pytest -q tests/test_s9_1_output_contract.py
uv run pytest -q tests/test_s9_2_errors.py
uv run pytest -q tests/test_s9_3_openapi.py
uv run pytest -q tests/test_s9_4_student.py

# Contrato y calidad
uv run python -m wine_api.export_openapi
uv run ruff check .
uv run ruff format --check .

# API local (Bash) y modelo no disponible
uv run fastapi dev
WINE_API_MODE=unavailable uv run fastapi dev

# PowerShell
$env:WINE_API_MODE="unavailable"; uv run fastapi dev
```

9. Notas y limites

- No modifiques predictor.py, request_id.py, dependencies.py ni los tests de S9.1-S9.3; si un test parece incorrecto, coméntalo con el docente.

- No añadas autenticación ni API keys: la protección de endpoints llega en S10 y estos tests serán su red de seguridad.

- Sigue la skill de FastAPI del repositorio: tipos de retorno, Annotated, parámetros comunes en el APIRouter y def para código bloqueante.

- La solución de referencia se consulta después de la entrega.

Referencia en el repositorio

semana9/modules/09-api-professional/exercises/02-wine-api-hardening/README.md
semana9/modules/09-api-professional/exercises/02-wine-api-hardening/problem/starter/README.md